

Write the Python program for Map Coloring to implement CSP

Aim

To implement the Map Coloring problem using the Constraint Satisfaction Problem (CSP) approach in Python.

Objective

To assign different colors to neighboring regions of a map such that no two adjacent regions have the same color.

Theory

A **Constraint Satisfaction Problem (CSP)** is a problem in which values are assigned to variables while satisfying a set of constraints. The Map Coloring problem is a classic example of CSP, where each region on a map is considered a variable, the available colors form the domain, and the constraint is that no two adjacent regions should have the same color. CSP algorithms use backtracking and constraint checking to find a valid color assignment efficiently.

Algorithm

1. Define the map and its neighboring regions.
2. Define the available colors.
3. Assign colors to each region one by one.
4. Check whether the selected color satisfies the CSP constraints.
5. If valid, assign the color.
6. Repeat until all regions are colored.
7. Display the final color assignment.

Flowchart

Start

|

Define Map & Colors

|

Select Region

|

Assign Color

|

Check Constraints

|

└──Yes──┘

| |

Assign Try Next Color

| |

Next Region

|

All Regions Colored?

|

Yes

|

Display Solution

|

Stop

Python Code

```
graph = {
```

```
    'A': ['B', 'C'],
```

```
    'B': ['A', 'C', 'D'],
```

```
    'C': ['A', 'B', 'D'],
```

```
    'D': ['B', 'C']
```

```
}
```

```
colors = ['Red', 'Green', 'Blue']
```

```
result = {}
```

```
def is_safe(node, color):  
    for neighbour in graph[node]:  
        if neighbour in result and result[neighbour] == color:  
            return False  
    return True
```

```
def solve(nodes, index):  
    if index == len(nodes):  
        return True
```

```
    node = nodes[index]
```

```
    for color in colors:  
        if is_safe(node, color):  
            result[node] = color  
            if solve(nodes, index + 1):  
                return True  
            result.pop(node)
```

```
    return False
```

```
nodes = list(graph.keys())
```

```
if solve(nodes, 0):  
    print("Color Assignment:")  
    for i in result:  
        print(i, "->", result[i])
```

```
else:  
    print("No Solution")
```

Sample Output

Color Assignment:

A -> Red

B -> Green

C -> Blue

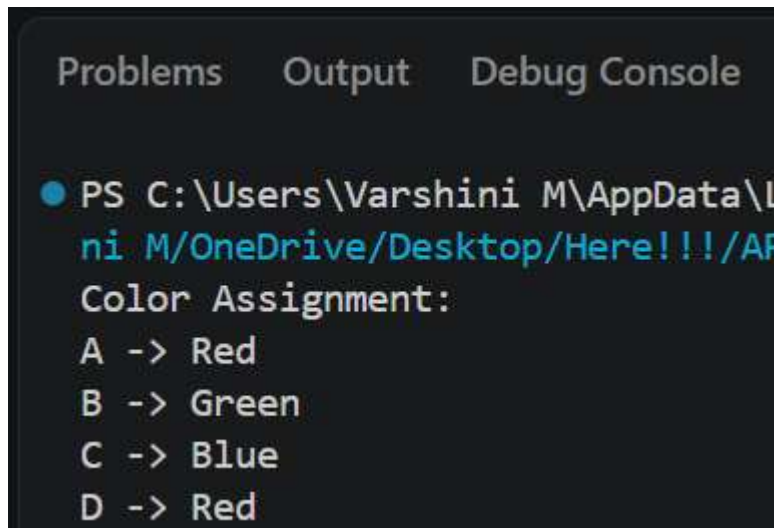
D -> Red

Result

The map coloring problem was solved successfully using the CSP approach.

Conclusion

The Constraint Satisfaction Problem efficiently assigns colors to regions while satisfying adjacency constraints.



```
Problems  Output  Debug Console

● PS C:\Users\Varshini M\AppData\Local\Microsoft\OneDrive\Desktop\Here!!!\AR
Color Assignment:
A -> Red
B -> Green
C -> Blue
D -> Red
```

C: > Users > Varshini M > OneDrive > Desktop > Here!!! > ARTIFICIAL INTELLIGENCE > VSC EXPs >

```
1 graph = {
2     'A': ['B', 'C'],
3     'B': ['A', 'C', 'D'],
4     'C': ['A', 'B', 'D'],
5     'D': ['B', 'C']
6 }
7
8 colors = ['Red', 'Green', 'Blue']
9 result = {}
10
11 def is_safe(node, color):
12     for neighbour in graph[node]:
13         if neighbour in result and result[neighbour] == color:
14             return False
15     return True
16
17 def solve(nodes, index):
18     if index == len(nodes):
19         return True
20
21     node = nodes[index]
22
23     for color in colors:
24         if is_safe(node, color):
25             result[node] = color
26             if solve(nodes, index + 1):
27                 return True
28             result.pop(node)
29
30     return False
31
32 nodes = list(graph.keys())
33
34 if solve(nodes, 0):
35     print("Color Assignment:")
36     for i in result:
37         print(i, "->", result[i])
38 else:
39     print("No Solution")
40
```